PROGRAMAÇÃO

Escola Naval

António Gonçalves

AGENDA DA AULA

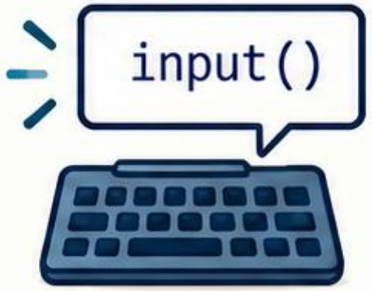
Programação • Escola Naval

1 Variáveis e tipos



Guardar e identificar valores

2 Entrada e conversão



Ler dados com input()

3 Saída e sequência



Apresentar resultados com print()

4 Expressões e operadores



Calcular com regras claras

5 Programa completo



Aplicar e testar



Conceito →  **exemplo** →  **aplicação** →  **verificação**

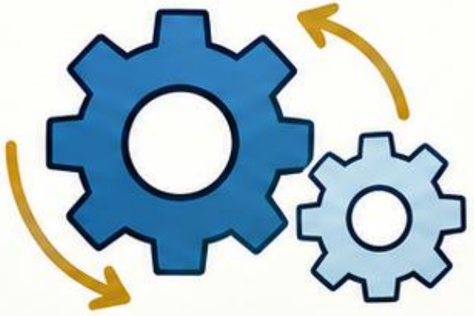
ENTRADA → PROCESSAMENTO → SAÍDA

ENTRADA



Dados recebidos

PROCESSAMENTO



Operações e cálculos

SAÍDA



Resultados apresentados

Exemplo com valores definidos no código

```
distancia = 120
velocidade = 15
```

```
tempo = distancia / velocidade
```

```
print(tempo)
8.0
```

Um programa transforma dados de entrada em resultados.

Variáveis e atribuição

Definição | Uma variável associa um nome a um valor.

Anatomia de uma variável



Exemplo em Python

```
nome = "Ana"    altura = 1.68    aprovado = True
```

ATUALIZAÇÃO E RASTREIO

Exemplo de código (Python)

```
numero = 10
numero = 20
numero = numero + 5
```

Tabela de rastreio

ETAPA	INSTRUÇÃO EXECUTADA	VALOR DE "numero"
Etapa 1	`numero = 10` ✔	10
Etapa 2	`numero = 20` →	20
Etapa 3	`numero = numero + 5` →	25



IDEIA-CHAVE: O valor atual resulta das instruções já executadas.

NOMES DE VARIÁVEIS EM PYTHON

Escolhe nomes válidos, claros e significativos.

NOMES VÁLIDOS

Seguem as regras e são usados corretamente.

`idade`

Nome simples e claro.

`nome_aluno`

Usa sublinhado para separar palavras.

`nota_final`

Descritivo e fácil de perceber.

NOMES INVÁLIDOS

Não seguem as regras do Python.

123

`2idade`

Começa por número.

`nome aluno`

Contém espaço.

`nota-final`

Contém hífen.

`class`

Palavra reservada da linguagem.

REGRA PRINCIPAL

Letras, algarismos e sublinhado.
O primeiro carácter não pode ser um algarismo.

MAIÚSCULAS E MINÚSCULAS DISTINGUEM NOMES

São variáveis diferentes.

`idade = 16`



`Idade = 20`



Preferir nomes claros: `velocidade_media` em vez de `v.`

TIPOS DE DADOS

PYTHON NA ESCOLA NAVAL

int — números inteiros



Números sem parte decimal.

20 **0** **-5**

Exemplo: 20 é um número inteiro (`int`).

float — números com parte decimal



Números reais, com parte decimal.

1.75 **12.5** **-0.5**

Exemplo: 1.75 é um número com parte decimal (`float`).



str — texto



Sequências de caracteres (letras, números, símbolos).

"Ana" **"20"**

Exemplo: "20" é texto (str), não é um número.

bool — valores lógicos



Representa valores de verdade.

True **False**

Exemplo: True e False são valores lógicos (bool).



O tipo do valor determina as operações possíveis.

type() – VERIFICAR O TIPO

Exemplo de código

```
idade = 16
altura = 1.72
nome = "Ana"
aprovado = True

print(type(idade))
print(type(altura))
print(type(nome))
print(type(aprovado))
```

A INSPECÇÃO COM type()



RESULTADOS DA INSPECÇÃO

Variável	Tipo de Dados (Output)
idade	→ <class 'int'>
altura	→ <class 'float'>
nome	→ <class 'str'>
aprovado	→ <class 'bool'>

ATENÇÃO

type() apenas informa; não altera o valor.

IDEIA-CHAVE

type() permite confirmar como Python está a interpretar um valor.

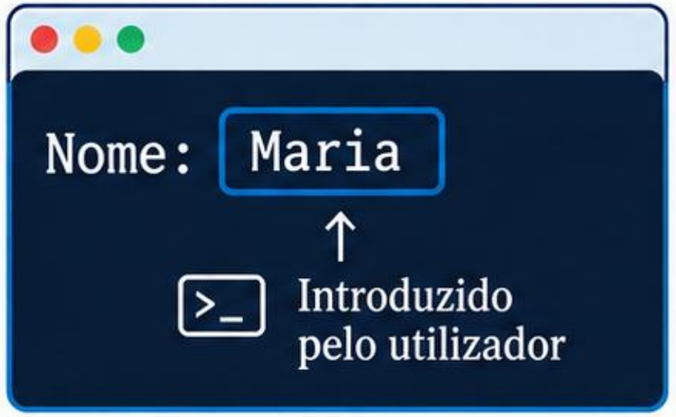
O QUE É input()?

```
nome = input("Nome: ")
```

input()

- apresenta uma mensagem;
- espera pela resposta;
- devolve aquilo que o utilizador escreveu.

Terminal



Resultado



→ "Maria"

nome

IDEIA-CHAVE

`input()` é a função principal para ler dados introduzidos pelo utilizador.

input() DEVOLVE SEMPRE TEXTO

Explicação

A função **input()** lê uma entrada do utilizador através do teclado e **devolve sempre um texto.**

Entrada do Utilizador	Execução	RESULTADO
 Entrada do Utilizador	EXECUÇÃO `idade = input("Idade: ")` Qual é o tipo de **idade**?	`idade = "16"` tipo: **str**

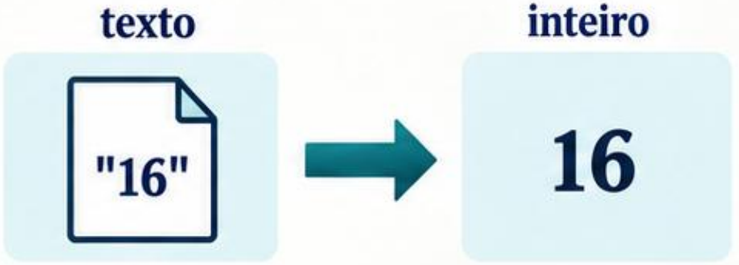
Conclusão | Mesmo quando o utilizador escreve números, **input()** devolve **texto.**

CONVERSÃO DE TIPOS

A conversão cria um valor do tipo pretendido quando o conteúdo é compatível.

1 DE TEXTO PARA INTEIRO

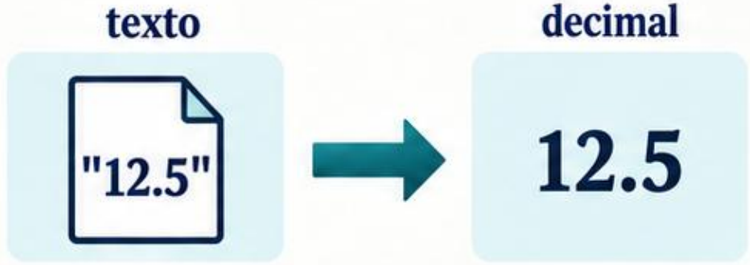
Converte uma cadeia de caracteres num número inteiro.




```
>>> int("16")
# resultado
16
```

2 DE TEXTO PARA DECIMAL

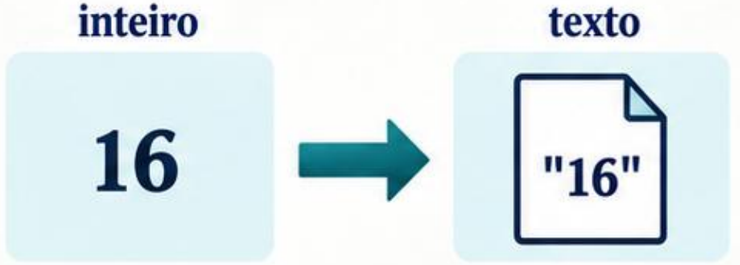
Converte uma cadeia de caracteres num número decimal (ponto flutuante).




```
>>> float("12.5")
# resultado
12.5
```

3 DE INTEIRO PARA TEXTO

Converte um número inteiro numa cadeia de caracteres.




```
>>> str(16)
# resultado
"16"
```

ESCOLHA PRÁTICA

Contagens: `int()`

Medidas: `float()`

Mensagens: `str()`

ERROS DE CONVERSÃO

Conversões válidas

`int("16")` → 16

`float("16.5")` → 16.5

Conversões inválidas

`int("dezasseis")` → ValueError

`int("16.5")` → ValueError

`float("16,5")` → ValueError

O texto tem de ser compatível com o tipo pretendido.

Nos números decimais, Python usa ponto: 16.5

LER E CONVERTER NUMA INSTRUÇÃO

Qual função é executada primeiro?

1. input("Idade: ")	2. utilizador escreve "16"	3. input() devolve "16"	4. int("16")	5. resultado = 16	6. idade recebe 16
`idade = int(input("Idade: "))`			16 → `int("16")` → 16	**16**	`idade = 16`
A função **input()** é executada primeiro. Mostra "**Idade:** " no ecrã e espera a resposta do utilizador.	O utilizador escreve "16".	A função **input()** devolve a cadeia de caracteres "16".	A função **int()** converte a cadeia de caracteres "16" no número inteiro 16.	O resultado da conversão é 16 (o número inteiro).	A variável **idade** recebe o valor 16.

IDEIA-CHAVE



Numa expressão combinada, a leitura ocorre antes da conversão.

SAÍDA COM print()

Código Python

```
print("Olá")
print(42)
nome = "Ana"
print("Nome:", nome)
```

Saída no ecrã



Apresenta texto e valores

A vírgula separa argumentos

Por defeito, separa argumentos com espaços e termina a linha

TEXTO, VARIÁVEIS E EXPRESSÕES

```
nome = "Ana"
```

	Instrução	Saída
Texto literal	`print("nome")`	`nome`
Valor da variável	`print(nome)`	`Ana`
Expressão calculada	`print(10 + 5)`	`15`
Texto literal	`print("10 + 5")`	`10 + 5`

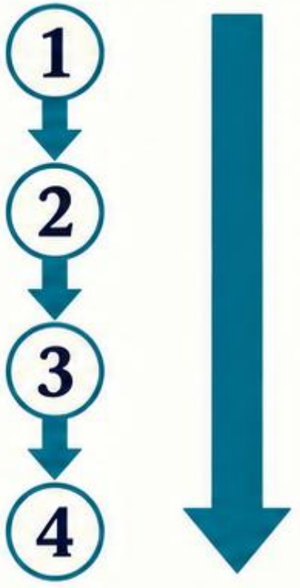
As aspas distinguem texto literal de código a avaliar.

O QUE É EXECUÇÃO SEQUENCIAL?

Código Python (instruções)

```
a = 10
b = 5
resultado = a + b
print(resultado)
```

Ordem de execução



Saída do programa



Nesta fase, as instruções são executadas uma após outra.

IDEIA-CHAVE: Num programa sequencial, Python executa as instruções de cima para baixo.

Rastrear várias variáveis

Código Python

```
1  x = 5
2  y = 10
3  z = x + y
4  x = x + 10
```

Estado das variáveis após cada instrução

	x	y	z
→ Após instrução `x = 5`	x = 5	y = —	z = —
→ Após instrução `y = 10`	x = 5	y = 10	z = —
→ Após instrução `z = x + y`	x = 5	y = 10	z = 15
→ Após instrução `x = x + 10`	x = 15	y = 10	z = 15



Porque z continua 15?

EXPRESSÕES, OPERADORES E OPERANDOS

Expressão	Elementos da expressão	Resultado
10 + 5 expressão	**10** (operando) **+** (operador) **5** (operando)	**15** resultado



Uma expressão combina valores e operadores para produzir um resultado.

IDEIA-CHAVE: Expressão = operandos + operadores → resultado.



OPERADORES ESPECIAIS: //, %, **

// | Divisão inteira

Devolve o **quociente** da divisão inteira, ignorando o resto.

Exemplo:

17 // 5 → 3

quociente

17 = 3 × 5 + 2

% | Resto

Devolve o resto da divisão inteira.

Exemplo:

17 % 5 → 2

resto

17 = 3 × 5 + 2

** | Potência

Eleva o primeiro operando à potência do segundo.

Exemplo:

2 ** 3 → 8

2 × 2 × 2 = 8

Nota importante:
Arredonda o quociente para baixo.

−7 // 2 = −4

Tipos e Operadores

ESCOLA NAVAL

Conceito	Exemplo em Python	Resultado	Explicação
Soma numérica	`10 + 5`	→ `15`	Soma dois números e devolve um resultado numérico.
Concatenação de texto	`"10" + "5"`	→ `"105"`	Junta duas cadeias de texto e devolve um único texto.
Repetição de texto	`"Python" * 2`	→ `"PythonPython"`	Repete o texto o número indicado de vezes.
Tipos incompatíveis	`"10" + 5`	→ `TypeError`	Não é possível somar uma cadeia de texto com um número.
Operação numérica mista	`10 + 2.5`	→ `12.5`	Soma um número inteiro com um número decimal.

O comportamento de um operador depende dos tipos dos operandos.

Precedência e Parênteses

A ordem das operações e o uso de parênteses determinam o resultado.

1 Parênteses ()

As expressões entre parênteses são calculadas primeiro.



```
2 + 3 * 4 = 14
    ↓
3 * 4 = 12
```

2 Potência **

A potência tem precedência sobre multiplicação, divisão, adição e subtração.



```
2 ** 3 = 8
```

3 Multiplicação e divisão: * / // %

A multiplicação, a divisão, a divisão inteira (//) e o resto da divisão (%) têm a mesma precedência.



```
(2 + 3) * 4 = 20
   ↓
2 + 3 = 5
```

4 Adição e subtração: + −

A adição e a subtração têm a mesma precedência.



```
2 + 3 - 5 = 0
```



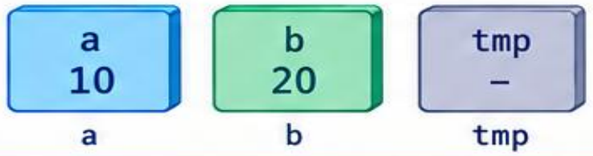
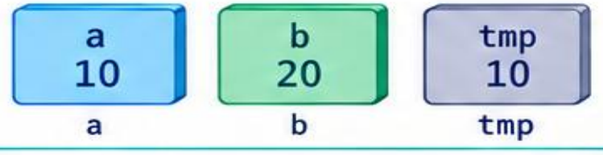
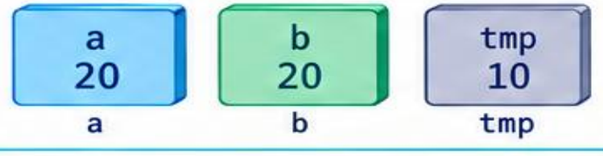
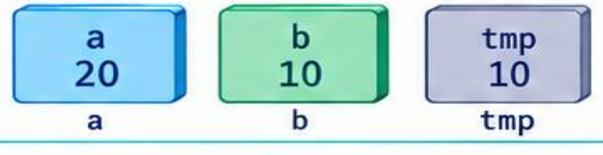
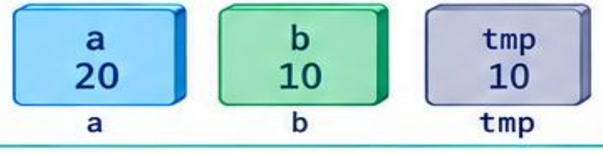
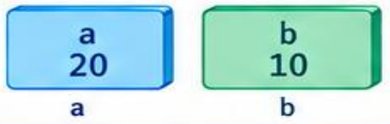
Ao mesmo nível, calcular da esquerda para a direita.

A potência associa da direita para a esquerda.

```
2 ** 3 ** 2 = 2 ** (3 ** 2) = 512
```

Troca de Valores entre Variáveis

PYTHON

Etapa	Código Python	Estado das variáveis	Explicação
1. Inicialização das variáveis	`a = 10` `b = 20`		Inicialmente, as variáveis a e b têm os valores 10 e 20, respetivamente. A variável tmp (temporária) ainda não tem valor.
2. Guardar o valor de a na variável temporária	`tmp = a`		O valor de a é guardado na variável tmp, de forma temporária, para não se perder.
3. Atribuir o valor de b a a	`a = b`		O valor de b é atribuído a a. Agora a tem o valor de 20, mas b ainda mantém o valor original.
4. Restaurar o valor de tmp em b	`b = tmp`		O valor de tmp é agora atribuído a b. A troca de valores está concluída.
5. Resultado	`print(a, b)` `# 20 10`		As variáveis a e b ficaram trocadas: a = 20 e b = 10. A variável tmp mantém o valor 10 (até ser reutilizada, se necessário).
Alternativa em Python	`a, b = b, a`		Em Python, é possível trocar os valores diretamente, sem variável auxiliar. A atribuição é feita de forma simultânea.

Para trocar os valores de duas variáveis, é necessário garantir que nenhum deles se perca durante a operação.

Do problema ao programa

Exercício final: Duração de uma viagem

Um navio viaja a velocidade constante. Calcular a duração em horas.

ENTRADA



distancia: milhas náuticas
velocidade: nós

PROCESSAMENTO



```
tempo = distancia / velocidade
```

SAÍDA



Tempo de viagem em horas.

Escrever um programa que leia os dois valores com `float(input(...))`, calcule tempo e apresente o resultado com `print()`.

Considerar distância não negativa e velocidade maior que zero.

Teste: 120 milhas náuticas e 15 nós

Resultado esperado: 8.0 horas